

SENTINEL-GNSS — Colab Training Notebook

Before running: Runtime ▶ Change runtime type ▶ T4 GPU

What lives where	
Code + all data CSVs	GitHub (cloned in Step 1)
Feature windows (.npz)	Built fresh each session from CSV
Checkpoints + figures	Google Drive (survives disconnects)

If Colab disconnects mid-training: Re-run Steps 1–3, then run the recovery cell at the bottom — it resumes from the last saved checkpoint automatically.

Step 1 — Clone repo from GitHub

All code and processed CSVs (including the labelled dataset) are pulled directly from GitHub.

```
import os; os.chdir('/content')
```

```
import os, shutil

GITHUB_REPO = 'https://github.com/Jorshuare/AI-Based-Prediction-for-GNSS-Signal-Degradation.git'
REPO_DIR    = '/content/sentinel-gnss'

# Always anchor to /content first so the kernel is never left in a deleted directory
%cd /content

if os.path.exists(f'{REPO_DIR}/.git'):
    print('Repo already cloned – pulling latest ...')
    %cd {REPO_DIR}
    !git pull
else:
    # Remove stale directory if it exists without .git (e.g. leftover from failed clone)
    if os.path.exists(REPO_DIR):
        print('Removing stale directory ...')
        shutil.rmtree(REPO_DIR)
    print('Cloning repo ...')
    !git clone {GITHUB_REPO} {REPO_DIR}
    %cd {REPO_DIR}

print(f'\nWorking directory: {os.getcwd()}')
!git log --oneline -5

# Confirm labelled CSV came down with the repo
csv_path = f'{REPO_DIR}/data/labelled/sentinel_gnss_labelled.csv'
assert os.path.exists(csv_path), f'CSV not found at {csv_path}'
import pandas as pd
df = pd.read_csv(csv_path)
print(f'\nDataset: {len(df):,} rows × {len(df.columns)} columns – ready.')
```

```
/content
Cloning repo ...
Cloning into '/content/sentinel-gnss'...
remote: Enumerating objects: 377, done.
remote: Counting objects: 100% (54/54), done.
remote: Compressing objects: 100% (42/42), done.
```

```
remote: Total 377 (delta 21), reused 35 (delta 11), pack-reused 323 (from 1)
Receiving objects: 100% (377/377), 260.03 MiB | 30.20 MiB/s, done.
Resolving deltas: 100% (100/100), done.
Updating files: 100% (218/218), done.
/content/sentinel-gnss

Working directory: /content/sentinel-gnss
9531888 (HEAD -> main, origin/main, origin/HEAD) fix: train without smote"
de647d6 fix: softer class weights [1,2,3], remove titles, fix saliency backward, add __init__.py
32892ed fix: softer class weights [1,2,3], remove titles, fix saliency backward, add __init__.py
08d3416 fix: softer class weights [1,2,3], remove titles, fix saliency backward, add __init__.py
c50a86e fix: add __init__.py to make src a proper Python package

Dataset: 97,393 rows x 41 columns — ready.
```

Step 2 — Install extra dependencies + verify GPU

Colab already ships with PyTorch, NumPy, pandas, scikit-learn, matplotlib, seaborn, scipy.

Only `imbalanced-learn` (SMOTE) needs installing.

```
!pip install -q imbalanced-learn

import torch
print(f'PyTorch : {torch.__version__}')
print(f'CUDA    : {torch.cuda.is_available()}')
if torch.cuda.is_available():
    props = torch.cuda.get_device_properties(0)
    print(f'GPU      : {torch.cuda.get_device_name(0)}')
    print(f'VRAM      : {props.total_memory / 1e9:.1f} GB')
else:
    print('WARNING: No GPU — go to Runtime > Change runtime type > T4 GPU')

PyTorch : 2.10.0+cpu
CUDA    : False
WARNING: No GPU — go to Runtime > Change runtime type > T4 GPU
```

Step 3 — Mount Google Drive (checkpoints + figures only)

Checkpoints are saved to Drive so training survives disconnects.

Data CSVs are **not** needed here — they came from GitHub in Step 1.

```
from google.colab import drive
import os, shutil

drive.mount('/content/drive')

REPO_DIR = '/content/sentinel-gnss'
DRIVE_BASE = '/content/drive/MyDrive/sentinel-gnss'

# Folders that must persist across sessions
PAIRS = [
    (f'{REPO_DIR}/results/models/checkpoints', f'{DRIVE_BASE}/checkpoints'),
    (f'{REPO_DIR}/results/figures', f'{DRIVE_BASE}/figures'),
]

for local, on_drive in PAIRS:
    os.makedirs(on_drive, exist_ok=True)
    os.makedirs(os.path.dirname(local), exist_ok=True)
```

```

if os.path.isdir(local) and not os.path.islink(local):
    shutil.rmtree(local) # remove empty local dir
if not os.path.islink(local):
    os.symlink(on_drive, local)
print(f' {os.path.basename(local):15s} → {on_drive}')

# Report any existing checkpoints (from a previous run)
ckpt_dir = f'{DRIVE_BASE}/checkpoints'
ckpts = sorted(f for f in os.listdir(ckpt_dir) if f.endswith('.pt'))
if ckpts:
    print(f'\nExisting checkpoints on Drive ({len(ckpts)}):')
    for c in ckpts:
        print(f' {c}')
    print('\nRun Step 5 with --resume to continue training.')
else:
    print('\nNo existing checkpoints – will start fresh.')

```

```

Mounted at /content/drive
checkpoints → /content/drive/MyDrive/sentinel-gnss/checkpoints
figures     → /content/drive/MyDrive/sentinel-gnss/figures

```

```
No existing checkpoints – will start fresh.
```

✓ Step 4 — Build feature windows

Reads `data/labelled/sentinel_gnss_labelled.csv` → produces sliding-window tensors.

Skips automatically if windows already exist (safe to re-run).

```

%cd /content/sentinel-gnss
!python -m src.models.feature_prep

# Sanity check
import numpy as np
print('\nWindow shapes:')
for split in ('train', 'val', 'test'):
    d = np.load(f'data/processed/windows/{split}.npz')
    print(f' {split:5s} X={d["X"].shape} labels(5s): CLEAN={np.sum(d["y_5s"]==0):,} WARNING={np.sum(d["y_5s"]==1):,} DEGRADED={np.sum(d["y_5s"]==2):,}')

```

```

/content/sentinel-gnss
INFO Loading /content/sentinel-gnss/data/labelled/sentinel_gnss_labelled.csv ...
INFO 97,393 rows x 41 columns
INFO label dist: {0: 73370, 1: 15075, 2: 8948}
INFO split dist: {'train': 53077, 'val': 26525, 'test': 17791}
INFO Feature columns (34): ['alt', 'baseline_sats', 'clock_bias', 'cnr_trend', 'cnr_variance', 'cycle_slips', 'dop_ratio', 'elevation_violations', 'fix_continuity', 'fix_transitions', 'gdop', 'hdop', '
INFO Scaler saved → /content/sentinel-gnss/data/processed/scaler.pkl
WARNING Session 'scenario_a_r1' too short; skipping.
WARNING Session 'scenario_a_r2' too short; skipping.
INFO train: 52,757 windows | 5s label dist: [35223 10600 6934]
INFO val : 23,842 windows | 5s label dist: [23035 444 363]
INFO test : 17,803 windows | 5s label dist: [13740 3003 1060]
INFO Applying SMOTE to training set ...
INFO SMOTE input shape: (52757, 1020) | labels: [35223 10600 6934]
INFO SMOTE output shape: (105669, 1020) | labels: [35223 35223 35223]
INFO Saved train windows → /content/sentinel-gnss/data/processed/windows/train.npz (105,669 samples)
INFO Saved val windows → /content/sentinel-gnss/data/processed/windows/val.npz (23,842 samples)
INFO Saved test windows → /content/sentinel-gnss/data/processed/windows/test.npz (17,803 samples)

```

```

=== Window summary ===
train X=(105669, 30, 34) y_5s=[35223 35223 35223]
val   X=(23842, 30, 34) y_5s=[23035 444 363]
test  X=(17803, 30, 34) y_5s=[13740 3003 1060]

```

```
Window shapes:
train X=(105669, 30, 34) labels(5s): CLEAN=35,223 WARNING=35,223 DEGRADED=35,223
val   X=(23842, 30, 34) labels(5s): CLEAN=23,035 WARNING=444 DEGRADED=363
test  X=(17803, 30, 34) labels(5s): CLEAN=13,740 WARNING=3,003 DEGRADED=1,060
```

```
!pip install -q xgboost
!python -m src.models.baselines
```

```
INFO Loading windows ...
INFO Train: (105669, 1020) Test: (17803, 1020)
INFO
--- Tier 1: Majority-class -----
INFO
--- Tier 2: C/N0 threshold rule -----
INFO
--- Tier 3: Random Forest -----
INFO RandomForest +5s MacroF1=0.5433 MCC=0.4159
INFO RandomForest +15s MacroF1=0.5320 MCC=0.4010
INFO RandomForest +30s MacroF1=0.5198 MCC=0.3827
INFO
--- Tier 3: XGBoost -----
INFO XGBoost +5s MacroF1=0.5538 MCC=0.4280
INFO XGBoost +15s MacroF1=0.5329 MCC=0.4051
INFO XGBoost +30s MacroF1=0.5207 MCC=0.3904
INFO

INFO =====
INFO Comparison Table – Test Set Macro-F1 (higher is better)
INFO =====
INFO Method Justification +5s +15s +30s
INFO -----
INFO MajorityClass Trivial floor 0.2904 0.0377 0.0380
INFO CNR_Threshold Domain rule (RTCM) 0.0375 0.0377 0.0380
INFO RandomForest Classical ML baseline 0.5433 0.5320 0.5198
INFO XGBoost Classical ML baseline 0.5538 0.5329 0.5207
INFO =====
INFO Macro-F1: equally weights CLEAN / WARNING / DEGRADED classes.
INFO Random 3-class predictor would score ≈ 0.3333.
INFO =====
INFO
Full metrics saved → /content/sentinel-gnss/results/baselines/baseline_comparison.json
```

✓ Step 5 — Train

`--resume` continues from the last Drive checkpoint if it exists, otherwise starts fresh.

Checkpoints are saved directly to Drive every 10 epochs + whenever a new best val F1 is reached.

```
import shutil, os, glob

# Wipe ALL checkpoints from Drive so class-weight change takes full effect
drive_ckpt = '/content/drive/MyDrive/sentinel-gnss/checkpoints'
local_ckpt = '/content/sentinel-gnss/results/models/checkpoints'

for ckpt_path in [drive_ckpt, local_ckpt]:
    if os.path.exists(ckpt_path):
        for f in glob.glob(f'{ckpt_path}/*'):
            os.remove(f)
        print(f'Cleared: {ckpt_path}')

%cd /content/sentinel-gnss
# Fresh training run – class weights [1.0, 2.0, 3.0], batch_size=256 on T4
!python -m src.models.train --batch size 256
```

```

Cleared: /content/drive/MyDrive/sentinel-gnss/checkpoints
Cleared: /content/sentinel-gnss/results/models/checkpoints
/content/sentinel-gnss
INFO Device: cpu
INFO Loading windows ...
INFO train batches: 413 val batches: 94
/content/sentinel-gnss/src/models/transformer_lstm.py:204: UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False because encoder_layer.norm_first was True
  self.transformer = nn.TransformerEncoder(
SentinelGNSS | parameters: 359,241
INFO
=====
INFO Training SENTINEL-GNSS | 359,241 parameters
INFO Epochs: 0 → 150 | Patience: 20
INFO =====

INFO Epoch 001/150 | tr_loss=0.2898 val_loss=0.0690 | val_F1(5s/15s/30s): 0.756/0.700/0.669 | lr=4.00e-04 324.0s
INFO ★ New best val macro-F1 = 0.7083 → checkpoint_best.pt
INFO Epoch 002/150 | tr_loss=0.1827 val_loss=0.0605 | val_F1(5s/15s/30s): 0.771/0.727/0.685 | lr=6.00e-04 325.2s
INFO ★ New best val macro-F1 = 0.7277 → checkpoint_best.pt
INFO Epoch 003/150 | tr_loss=0.1532 val_loss=0.0474 | val_F1(5s/15s/30s): 0.779/0.743/0.707 | lr=8.00e-04 324.3s
INFO ★ New best val macro-F1 = 0.7431 → checkpoint_best.pt
INFO Epoch 004/150 | tr_loss=0.1361 val_loss=0.0513 | val_F1(5s/15s/30s): 0.801/0.779/0.724 | lr=1.00e-03 328.0s
INFO ★ New best val macro-F1 = 0.7679 → checkpoint_best.pt
INFO Epoch 005/150 | tr_loss=0.1284 val_loss=0.0539 | val_F1(5s/15s/30s): 0.793/0.754/0.705 | lr=1.00e-03 327.8s
INFO Epoch 006/150 | tr_loss=0.1185 val_loss=0.0504 | val_F1(5s/15s/30s): 0.804/0.775/0.734 | lr=1.00e-03 325.0s
INFO ★ New best val macro-F1 = 0.7710 → checkpoint_best.pt
INFO Epoch 007/150 | tr_loss=0.1121 val_loss=0.0558 | val_F1(5s/15s/30s): 0.785/0.761/0.711 | lr=1.00e-03 324.6s
INFO Epoch 008/150 | tr_loss=0.1045 val_loss=0.0653 | val_F1(5s/15s/30s): 0.778/0.752/0.711 | lr=9.99e-04 327.0s
INFO Epoch 009/150 | tr_loss=0.1028 val_loss=0.0556 | val_F1(5s/15s/30s): 0.792/0.765/0.722 | lr=9.98e-04 327.2s
INFO Epoch 010/150 | tr_loss=0.0974 val_loss=0.0676 | val_F1(5s/15s/30s): 0.774/0.734/0.696 | lr=9.97e-04 323.6s
INFO → Periodic checkpoint: checkpoint_epoch_010.pt
INFO Epoch 011/150 | tr_loss=0.0947 val_loss=0.0693 | val_F1(5s/15s/30s): 0.757/0.729/0.675 | lr=9.96e-04 323.1s
INFO Epoch 012/150 | tr_loss=0.0908 val_loss=0.0674 | val_F1(5s/15s/30s): 0.810/0.749/0.685 | lr=9.94e-04 324.1s
INFO Epoch 013/150 | tr_loss=0.0877 val_loss=0.0705 | val_F1(5s/15s/30s): 0.776/0.734/0.667 | lr=9.93e-04 325.5s
INFO Epoch 014/150 | tr_loss=0.0855 val_loss=0.0715 | val_F1(5s/15s/30s): 0.775/0.731/0.700 | lr=9.91e-04 325.7s
INFO Epoch 015/150 | tr_loss=0.0841 val_loss=0.0689 | val_F1(5s/15s/30s): 0.779/0.742/0.698 | lr=9.88e-04 323.0s
INFO Epoch 016/150 | tr_loss=0.0818 val_loss=0.0828 | val_F1(5s/15s/30s): 0.796/0.752/0.706 | lr=9.86e-04 326.7s
INFO Epoch 017/150 | tr_loss=0.0782 val_loss=0.0833 | val_F1(5s/15s/30s): 0.809/0.779/0.733 | lr=9.83e-04 329.4s
INFO ★ New best val macro-F1 = 0.7737 → checkpoint_best.pt
INFO Epoch 018/150 | tr_loss=0.0763 val_loss=0.0846 | val_F1(5s/15s/30s): 0.768/0.740/0.695 | lr=9.80e-04 327.5s
INFO Epoch 019/150 | tr_loss=0.0751 val_loss=0.0872 | val_F1(5s/15s/30s): 0.766/0.719/0.691 | lr=9.77e-04 324.6s
INFO Epoch 020/150 | tr_loss=0.0721 val_loss=0.0808 | val_F1(5s/15s/30s): 0.804/0.769/0.713 | lr=9.74e-04 322.1s
INFO → Periodic checkpoint: checkpoint_epoch_020.pt
INFO Epoch 021/150 | tr_loss=0.0711 val_loss=0.0766 | val_F1(5s/15s/30s): 0.800/0.754/0.709 | lr=9.70e-04 324.3s
INFO Epoch 022/150 | tr_loss=0.0699 val_loss=0.0919 | val_F1(5s/15s/30s): 0.787/0.733/0.693 | lr=9.66e-04 322.5s
INFO Epoch 023/150 | tr_loss=0.0670 val_loss=0.0806 | val_F1(5s/15s/30s): 0.805/0.747/0.723 | lr=9.62e-04 327.5s
INFO Epoch 024/150 | tr_loss=0.0682 val_loss=0.0775 | val_F1(5s/15s/30s): 0.783/0.749/0.697 | lr=9.58e-04 324.7s
INFO Epoch 025/150 | tr_loss=0.0646 val_loss=0.0853 | val_F1(5s/15s/30s): 0.808/0.741/0.716 | lr=9.54e-04 327.6s
INFO Epoch 026/150 | tr_loss=0.0649 val_loss=0.0873 | val_F1(5s/15s/30s): 0.792/0.756/0.693 | lr=9.49e-04 325.3s
INFO Epoch 027/150 | tr_loss=0.0637 val_loss=0.0871 | val_F1(5s/15s/30s): 0.813/0.764/0.706 | lr=9.44e-04 322.3s
INFO Epoch 028/150 | tr_loss=0.0614 val_loss=0.1005 | val_F1(5s/15s/30s): 0.763/0.717/0.672 | lr=9.39e-04 325.4s
INFO Epoch 029/150 | tr_loss=0.0618 val_loss=0.0856 | val_F1(5s/15s/30s): 0.783/0.752/0.699 | lr=9.34e-04 327.9s
INFO Epoch 030/150 | tr_loss=0.0591 val_loss=0.0864 | val_F1(5s/15s/30s): 0.811/0.769/0.721 | lr=9.28e-04 326.0s
INFO → Periodic checkpoint: checkpoint_epoch_030.pt
INFO Epoch 031/150 | tr_loss=0.0592 val_loss=0.0974 | val_F1(5s/15s/30s): 0.791/0.730/0.689 | lr=9.23e-04 325.3s
INFO Epoch 032/150 | tr_loss=0.0572 val_loss=0.1067 | val_F1(5s/15s/30s): 0.800/0.749/0.685 | lr=9.17e-04 326.0s
INFO Epoch 033/150 | tr_loss=0.0580 val_loss=0.0974 | val_F1(5s/15s/30s): 0.808/0.736/0.682 | lr=9.11e-04 323.9s
INFO Epoch 034/150 | tr_loss=0.0565 val_loss=0.1059 | val_F1(5s/15s/30s): 0.788/0.733/0.665 | lr=9.05e-04 325.1s

```

▼ Step 6 — Evaluate

Loads `checkpoint_best.pt` from Drive, runs all 14 analyses, saves figures to Drive.

```
%cd /content/sentinel-gnss
!python -m src.models.evaluate

+15s per-class breakdown:
  CLEAN      P=0.926  R=0.537  F1=0.680  n=13710
  WARNING    P=0.179  R=0.468  F1=0.259  n=3025
  DEGRADED   P=0.320  R=0.581  F1=0.412  n=1068

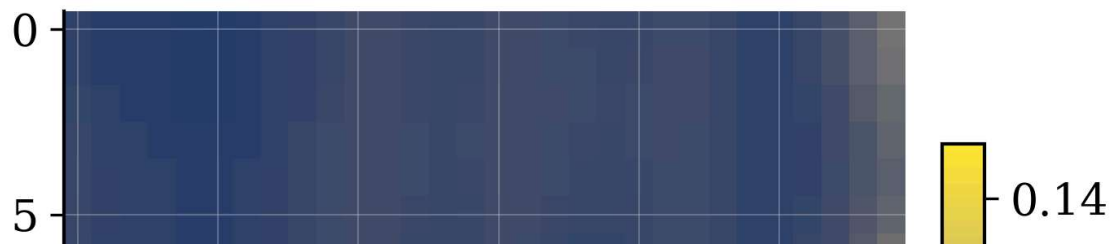
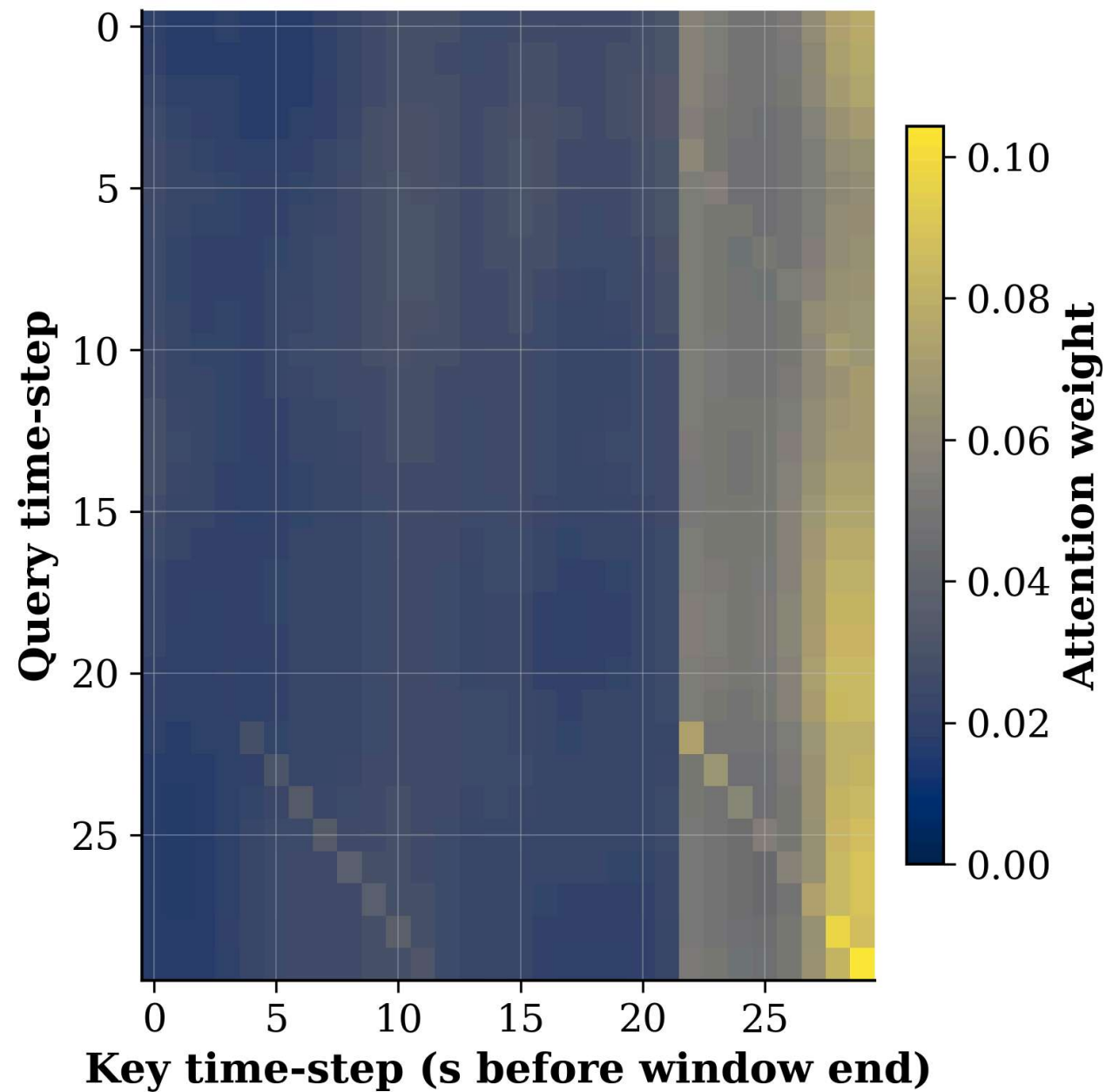
+30s per-class breakdown:
  CLEAN      P=0.935  R=0.534  F1=0.680  n=13667
  WARNING    P=0.174  R=0.445  F1=0.250  n=3061
  DEGRADED   P=0.242  R=0.483  F1=0.322  n=1075
```

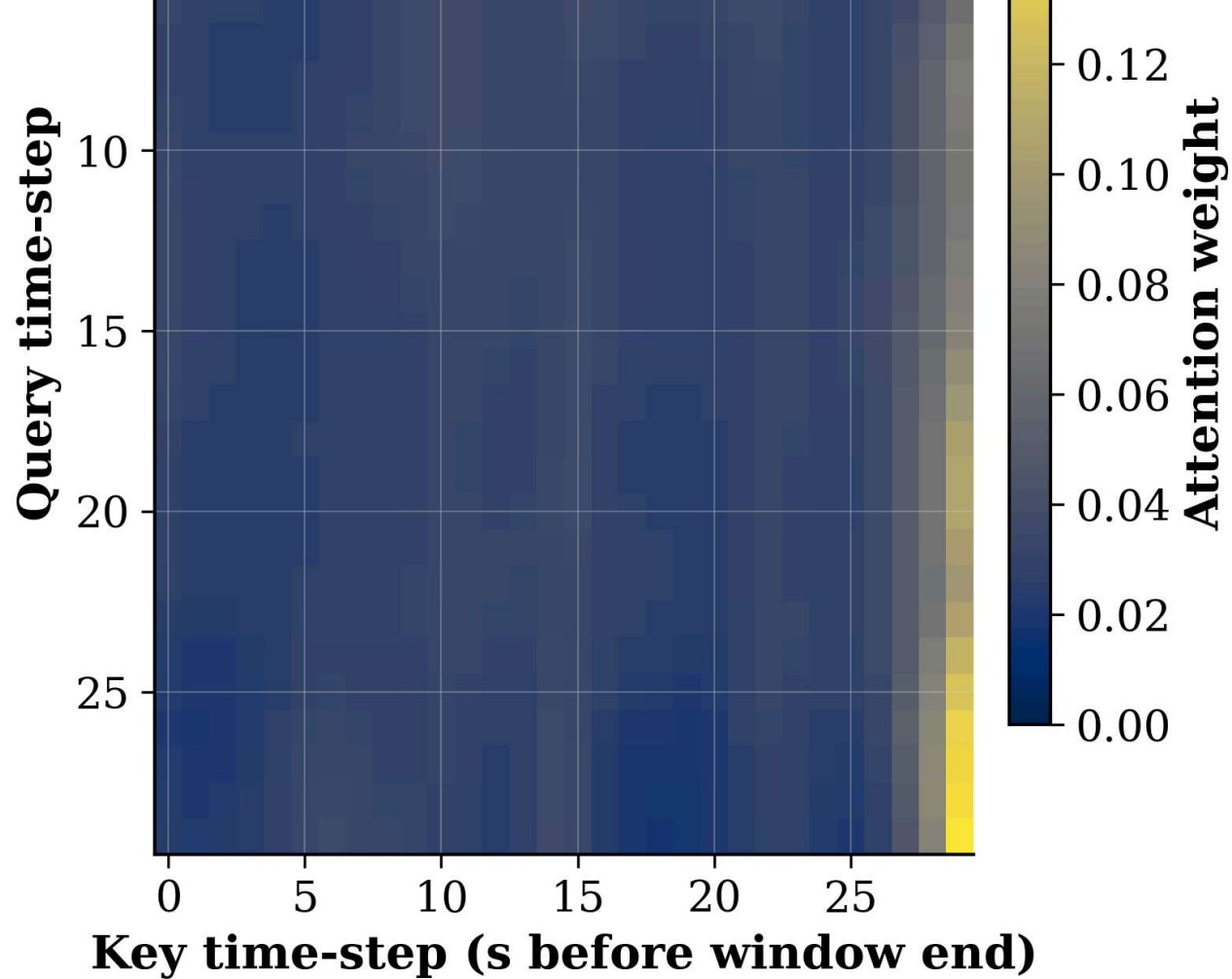
```
INFO  Metrics saved → /content/sentinel-gnss/results/figures/metrics_test.json
INFO  Computing 1000-iteration bootstrap CIs ...
INFO  Bootstrap CI (5s): macro-F1=[0.458, 0.477]  MCC=[0.224, 0.246]
INFO  Bootstrap CI (15s): macro-F1=[0.442, 0.460]  MCC=[0.206, 0.228]
INFO  Bootstrap CI (30s): macro-F1=[0.409, 0.427]  MCC=[0.191, 0.213]
INFO  Generating figures ...
  Saved → /content/sentinel-gnss/results/figures/confusion_matrices_test.pdf
  Saved → /content/sentinel-gnss/results/figures/confusion_matrices_test.png
INFO  ✓ Confusion matrices
  Saved → /content/sentinel-gnss/results/figures/roc_curves_test.pdf
  Saved → /content/sentinel-gnss/results/figures/roc_curves_test.png
INFO  ✓ ROC curves
  Saved → /content/sentinel-gnss/results/figures/pr_curves_test.pdf
  Saved → /content/sentinel-gnss/results/figures/pr_curves_test.png
INFO  ✓ Precision-Recall curves
  Saved → /content/sentinel-gnss/results/figures/calibration_curves_test.pdf
  Saved → /content/sentinel-gnss/results/figures/calibration_curves_test.png
INFO  ✓ Calibration curves
  Saved → /content/sentinel-gnss/results/figures/learning_curves.pdf
  Saved → /content/sentinel-gnss/results/figures/learning_curves.png
INFO  ✓ Learning curves
  Saved → /content/sentinel-gnss/results/figures/multi_horizon_comparison.pdf
  Saved → /content/sentinel-gnss/results/figures/multi_horizon_comparison.png
INFO  ✓ Multi-horizon comparison
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_clean.pdf
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_clean.png
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_warning.pdf
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_warning.png
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_degraded.pdf
  Saved → /content/sentinel-gnss/results/figures/attention_heatmap_degraded.png
INFO  ✓ Attention heatmaps
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_5s.pdf
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_5s.png
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_15s.pdf
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_15s.png
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_30s.pdf
  Saved → /content/sentinel-gnss/results/figures/feature_saliency_30s.png
INFO  ✓ Feature saliency maps
  Saved → /content/sentinel-gnss/results/figures/lead_time_histogram.pdf
  Saved → /content/sentinel-gnss/results/figures/lead_time_histogram.png
INFO  ✓ Lead-time histogram
INFO
=====
INFO  All evaluation artefacts saved to: /content/sentinel-gnss/results/figures
INFO  =====
```

Step 7 — View all figures inline

```
import glob
from IPython.display import Image, display

figs = sorted(glob.glob('/content/sentinel-gnss/results/figures/*.png'))
print(f'{len(figs)} figures saved to Drive:')
for f in figs:
    name = f.split('/')[-1]
    print(f'  {name}')
    display(Image(filename=f, width=950))
```



attention_heatmap_warning.png

